

MeshMind

Complete Project Context / Handoff Document

Purpose: Give another coding/engineering agent enough context to understand the backend, GraphQL layer, AI document-processing pipeline, current design decisions, and remaining work without relying on the previous chat history.

Project status: Backend architecture is largely established. Current work has focused on integrating GraphQL resolvers with the service/repository/guard layers, adding context-based identity helpers, finalizing schemas, and preparing Google/GitHub OAuth callback URLs.

1. Product Overview

MeshMind is a document-processing and AI-assisted knowledge platform. The core user journey is: a user creates a project, uploads files, the backend stores the files, an asynchronous AI pipeline extracts and summarizes document content, embeddings/summaries are stored in a vector database, and the project chatbot uses that knowledge for Retrieval-Augmented Generation (RAG).

The intended architecture separates synchronous application responsibilities from asynchronous AI/document processing. Go is the primary backend/API language, while the AI worker can use Python where it provides the strongest ecosystem for document extraction, LangChain-based orchestration, embeddings, and LLM workflows.

2. High-Level Architecture

Layer	Technology / Role	Responsibility
Client	Web/mobile frontend + GraphQL client	Authentication, projects, files, chats, reports, flowcharts and user-facing UI.
API / Backend	Go + gqlgen	GraphQL API, auth, authorization, business services, validation, repositories, file metadata and ap
Database	PostgreSQL	Users, projects, files, chats/messages, reports, flowcharts, relationships, status/properties and app
Object storage	AWS S3	Durable storage for uploaded source documents and potentially generated artifacts.
Queue	AWS SQS	Asynchronous handoff from upload/application events to document-processing workers.
AI worker	Python	Document extraction, chunking, summarization, embeddings and AI workflow orchestration.
AI framework	LangChain	Primarily used in the AI/document-processing side for extraction, summarization/RAG orchestration
Vector DB	Qdrant	Stores embeddings and searchable document knowledge for RAG.
LLM	Ollama / local inference	Local LLM inference for summarization/chat generation where configured.

3. Core Data / Domain Model

User - Identity and account owner. Users participate in projects, chats, messages, reports, flowcharts and other project-level actions.

Project - Primary workspace/container. A project groups files, chats, generated reports, flowcharts and associated knowledge.

File - Uploaded source document. Stored in object storage with metadata in PostgreSQL. Upload triggers asynchronous processing.

Chat - Conversation scoped to a project and potentially used as the source context for generated artifacts.

ChatMessage - Individual message belonging to a chat, with a sender/user relationship and message content/metadata.

Report - Generated or managed project artifact. Includes title, content, format, properties, timestamps and optional source chat.

ReportProperties - Report metadata including generatedBy, generatedByAI, status and optional sourceChat.

Flowchart - Generated project artifact containing a JSON data representation, optional generating user, AI flag, status and optional source chat.

4. Document-to-RAG Flow

- 1. Upload:** Client sends a GraphQL mutation / upload operation. Go validates the request and creates/updates file metadata.
- 2. Store:** The original file is persisted in AWS S3. PostgreSQL stores the file record and application metadata.
- 3. Enqueue:** A processing event/job is sent to AWS SQS so the upload request does not block on expensive document processing.
- 4. Extract:** The AI worker retrieves the source file and extracts text/content. Python is preferred here because of the mature document-processing ecosystem.
- 5. Chunk / normalize:** Extracted text is cleaned and split into meaningful chunks suitable for embedding and retrieval.
- 6. Summarize:** LangChain is intended to be part of the AI workflow for summarization and LLM orchestration.
- 7. Embed:** Document/chunk representations are converted into embeddings and written to Qdrant.
- 8. Chat / RAG:** When a user asks the project chatbot a question, relevant knowledge is retrieved from Qdrant and supplied to the LLM as context.
- 9. Artifacts:** The same project knowledge/chat context can support generated reports and flowcharts, whose structured metadata remains in PostgreSQL.

5. Backend Architecture

The Go backend is intentionally layered. GraphQL resolvers should remain thin: parse/validate GraphQL input, obtain authenticated identity/context, call the appropriate service, and map service/domain errors into GraphQL-visible errors.

Expected dependency direction: GraphQL resolver -> Service -> Guard/Validator + Repository -> PostgreSQL / external infrastructure.

Resolvers should not contain SQL or detailed business rules. Repositories own persistence operations. Services own use-case orchestration. Guards enforce authorization/resource ownership. Validators handle input/domain validation. Utilities handle cross-cutting helpers such as UUID parsing.

Component	What belongs here
GraphQL resolver	GraphQL-specific input/output handling; authentication context access; calling service methods.
Service	Business use cases and orchestration; calls repositories/guards/validators.
Repository	Database access and persistence queries.
Guard	Authorization and ownership checks for resources.
Validator	Input and domain validation.
Utils	Reusable helpers such as parsing IDs, common transformations.
App/container	Application dependency wiring; services, repositories, infrastructure clients.

6. GraphQL / gqlgen Context

The project uses **gqlgen v0.17.95**. Generated resolver files contain resolver methods whose implementations are expected to delegate into the application layer.

A recurring issue discussed in the project was whether a resolver input containing an ID plus other fields can uniquely identify a resource. The answer is: the ID should be the canonical resource identity; other fields are mutation/update payload. The service should load the resource by ID and then validate authorization/ownership before applying changes.

For authenticated operations, the preferred pattern is to derive the current user from the request context rather than trusting a client-provided user ID. Context helpers such as `GetUserFromContext` / `GetUserIDFromContext` were being introduced for this purpose.

Representative resolver pattern:

```
func (r *mutationResolver) CreateChatMessage(ctx context.Context, input model.CreateChatMessageInput)
(*model.ChatMessage, error) {
    chatID, err := parseUUID(input.ChatID)
    if err != nil { return nil, err }
    senderID, err := parseUUID(input.SenderID)
    if err != nil { return nil, err }
    message, err := r.App.Services.Chat.Create...
    ...
}
```

Important integration rule: If sender identity is authenticated, prefer the context-derived user ID instead of allowing an arbitrary senderID from the client. The exact choice depends on the schema/use case, but authorization must prevent impersonation.

The GraphQL schema has been split into multiple schema files by domain. This is a maintainability choice; `gqlgen` merges them during generation. Generated files should not be manually treated as the source of truth - schema and custom resolver implementations are.

7. Schema / Artifact Context

The project contains domain-specific GraphQL definitions for artifacts such as Flowchart and Report. The Flowchart model discussed was:

```
Flowchart: id, project, name, data (JSON), generatedBy (nullable user), generatedByAI, status, sourceChat
(nullable), createdAt, updatedAt.
```

The Report model discussed was:

```
Report: id, project, title, content, format, properties, createdAt, updatedAt. ReportProperties includes
report, generatedBy, generatedByAI, status and optional sourceChat.
```

Query patterns include fetching an artifact by ID and listing artifacts by project ID. Mutation patterns include create/update/delete-style operations as defined by each domain schema.

8. Authentication

Authentication is being designed around normal application auth plus OAuth providers. Google and GitHub callback URLs were identified as the next integration/setup step.

Key security principle: OAuth callback handling should establish/resolve the local User, issue the application's authenticated session/token, and redirect back to the frontend. Provider identity should not be treated as the application's authorization model by itself.

The exact callback URL values depend on the deployed environment and configured frontend/backend domains. Development and production should use separate OAuth application credentials/callbacks where appropriate.

9. Database / Migrations

PostgreSQL is the system of record for application metadata and relationships. The project uses Goose for migrations.

A migration rollback issue was encountered when attempting to drop the enum type **flowchart_status**: PostgreSQL reported that the type could not be dropped because other objects depended on it (SQLSTATE 2BP01).

This indicates that the down migration order needs to remove dependent columns/tables/constraints/functions before dropping the enum type. A safe migration strategy is to reverse dependency order: drop dependent objects first, then the enum type. Do not blindly add CASCADE unless the intended dependency deletion is fully understood.

10. File Service / Upload Responsibilities

The FileService was being implemented using repository, guard, database, utility and validator packages. The service is responsible for coordinating file-related business operations rather than performing all storage/database work itself.

The intended upload lifecycle is: validate request -> authorize project/user -> create or prepare file metadata -> upload/store object -> persist storage metadata -> enqueue processing job -> return file state to client. Exact transaction boundaries should be chosen carefully because S3 and PostgreSQL are separate systems.

If processing is asynchronous, file status should make the lifecycle explicit (for example pending/processing/processed/failed, subject to the actual enum design). The worker should update state or publish a completion event in a way that is idempotent.

11. Chat / Messaging Context

Chat is both a user-facing conversational feature and a potential source for generated artifacts. A ChatMessage belongs to a Chat and has a sender. The resolver must parse IDs and delegate message creation to the Chat service.

The question 'but then how will the message be sent?' was addressed by distinguishing GraphQL mutation from transport: the mutation persists the message and returns it. If the UI requires live delivery to other connected clients, a separate real-time mechanism (GraphQL subscriptions, WebSocket/SSE, or another event transport) is needed. A mutation alone does not automatically push a message to another browser.

For a first implementation, persistence + client refetch/polling is simpler. For true real-time chat, add subscriptions/events after the basic create/read flow is stable.

12. AI Architecture Decision

A major architecture discussion was whether to implement the entire AI stack in Go. The current direction is **not** to force everything into Go.

Recommended split: Go owns product/backend concerns (GraphQL, auth, users, projects, files, persistence, authorization, job submission). Python owns AI-heavy processing where libraries such as LangChain and document extraction tooling are more mature.

LangChain should be used primarily in the AI worker for document processing, summarization, retrieval orchestration and LLM chains where useful. It does not need to be inserted into ordinary CRUD/service code.

This keeps the Go API responsive and makes expensive AI work asynchronous and independently scalable.

13. Current Project State

Area	Status	Notes
Core Go backend structure	Mostly established	Layered service/repository/guard/validator approach is in place.
GraphQL schemas	Mostly established	Schemas have been split by domain; artifact types such as Report and Flowchart are defined.
Resolvers	Integration work in progress	Resolvers are being wired to services; generated code should remain thin.
Context identity helpers	Being set up	Use request context for authenticated user identity.
Project/File services	Established / under integration	Continue checking resolver -> service -> repository and authorization boundaries.
Chat/message flow	Partially established	Create message mutation persists through Chat service; real-time delivery is a separate concern.

Area	Status	Notes
OAuth	Next setup task	Google and GitHub callback URLs/configuration remain to be completed.
Document AI pipeline	Architecture defined	S3 -> SQS -> Python worker -> extraction/chunking/summarization/embeddings -> Qdrant -> RAG.
Qdrant/RAG	Planned architecture	Used as vector knowledge layer for chatbot retrieval.
Database migration cleanup	Needs attention	Enum dependency ordering caused down-migration failure.

14. Recommended Next Steps

1. **Finish Google OAuth:** configure provider credentials, backend callback route, frontend redirect, allowed origins and environment variables.
2. **Finish GitHub OAuth:** same flow; ensure provider callback exactly matches configured URL.
3. **Verify context authentication:** every protected resolver should derive the authenticated user from context and not trust arbitrary client user IDs.
4. **Audit resolver/service integration:** every resolver should delegate to the correct service and avoid business logic duplication.
5. **Audit authorization:** project-scoped resources must verify that the authenticated user has access before read/update/delete operations.
6. **Stabilize file upload:** make S3 metadata and PostgreSQL state transitions explicit and make the SQS processing job idempotent.
7. **Implement Python worker:** consume SQS jobs, retrieve S3 file, extract text, chunk, summarize with LangChain/LLM, embed and upsert into Qdrant.
8. **Define processing states:** make failed/retryable/permanent failure semantics explicit so the frontend can display accurate progress.
9. **Implement chatbot RAG:** embed/query the user question, retrieve relevant Qdrant chunks, build grounded prompt, call LLM, return answer with source references if desired.
10. **Decide real-time chat transport:** if live messaging is required, add GraphQL subscriptions/WebSockets or an equivalent event mechanism.
11. **Fix migration down-order:** remove objects depending on flowchart_status before dropping the enum.
12. **Add integration tests:** especially auth, project ownership, file upload, message creation and artifact access.

15. Engineering Rules for the Next Agent

1. Treat the GraphQL schema as the API contract and generated gqlgen files as generated output.
2. Keep resolvers thin; business logic belongs in services.
3. Never use a client-supplied user ID as proof of identity. Use authenticated context and authorization checks.
4. Use resource IDs as canonical identity. Other mutation fields are attributes/changes, not a substitute for the resource ID.
5. Do not put SQL in GraphQL resolvers.
6. Keep PostgreSQL as the application metadata source of truth; keep S3 as the source-document object store; keep Qdrant as the vector retrieval store.
7. Use SQS to decouple expensive AI processing from synchronous upload/API requests.
8. Use Python/LangChain where document/LLM processing benefits from its ecosystem; do not introduce Python merely for ordinary backend CRUD.

9. Make asynchronous processing idempotent and observable.

10. When changing generated GraphQL code, first check whether the correct fix belongs in the schema, resolver implementation, or gqlgen configuration and regenerate rather than permanently editing generated output.

16. Mental Model for the Whole System

Think of MeshMind as two connected planes:

Application plane: Client -> GraphQL -> Go services -> PostgreSQL/S3/SQS. This plane handles users, permissions, projects, files, chats and generated artifact metadata.

Knowledge/AI plane: SQS -> Python worker -> S3 -> extraction/chunking -> LangChain/LLM -> embeddings -> Qdrant -> retrieval -> LLM -> chatbot/artifact generation.

The bridge between them is the file/job metadata and project identity. PostgreSQL knows which file belongs to which project; S3 stores the bytes; SQS tells the worker what to process; Qdrant stores searchable semantic knowledge associated with that project/document; the Go API exposes the resulting application behavior to the client.

This separation is the central architectural idea an incoming agent should preserve unless there is a concrete reason to change it.

Handoff note: This document is reconstructed from the MeshMind project context available in the conversation history. It intentionally records architectural decisions, current status, known issues, and next steps rather than pretending that every implementation detail has already been verified in the current source tree.